Week 1 – Class 2

From Prompt → Controlled AI Systems

From user → AI engineer mindset

Today you will learn to:

- Understand what a prompt actually is
- Control model behavior with system + user roles
- Use structure and constraints to improve outputs
- Understand why systems feel stateful even when models are stateless

Class theme: we are not doing deep prompt engineering yet — we are understanding prompt fundamentals.

The 4 AI architectures everyone should know

High level only — depth comes in later weeks

- Level 1 — Prompt systems
- Level 2 — Retrieval systems (RAG)
- Level 3 — Tool-using systems (agents)
- Level 4 — AI workflows (orchestrated systems)

Most courses stop at Level 1. We will go all the way — but at the right pace.

Level 1 — Prompt systems

The simplest architecture

- User input → Prompt → LLM → Response
- Example: “Summarize this report in 3 bullet points”
- Fast and useful, but limited
- LLMs can hallucinate, forget context, and cannot access your private data by default

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Summarize Python in 2 bullet points."}]
)

print(response.choices[0].message.content)
```

Micro demo: pure prompt system

Levels 2–4 (high level only)

Today we understand WHEN they are used, not HOW they work

- RAG → when AI needs knowledge from PDFs, docs, or internal data
- Agents → when AI needs to use tools or execute actions
- Workflows → when multiple steps combine into a larger system
- These become the rest of the course roadmap

What is a prompt, really?

A prompt is not just the question

- Prompt = system message + user message + optional history/context
- The model predicts tokens from the full input context you send
- If information is missing from the prompt, the model cannot use it
- Prompt structure is the first control surface of modern AI systems

“If it’s not in the prompt, it doesn’t exist for the model.”

System role vs user role

System defines behavior. User defines task.

System message

- Controls tone
- Controls style
- Controls behavior

User message

- Defines the task
- Adds constraints
- Specifies audience/output

System = HOW | User = WHAT

Live comparison: with and without system role

Show how structure changes behavior and tone.

```
# Without system message
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Explain hallucination."}]
)

# With system message
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "system", "content": "You are a precise AI instructor."},
        {"role": "user", "content": "Explain hallucination in one sentence."}
    ]
)
```

Structured prompts beat vague prompts

Same topic. Better structure. Better output.

- Bad: “Tell me about hallucination.”
- Better: “Explain hallucination in one sentence using simple language.”
- Professional: “Explain hallucination in exactly one sentence for software engineers.”
- LLMs respond better when ambiguity is reduced

Key lesson: more tokens do not automatically mean better output. Better constraints do.

Prompt quality experiment

Run all three and compare tone, precision, and token usage.

```
prompts = [
    "Tell me about hallucination",
    "Explain hallucination in one sentence",
    "Explain hallucination in one sentence for software engineers"
]

for p in prompts:
    response = client.chat.completions.create(
        model="gpt-4o-mini",
        messages=[{"role": "user", "content": p}],
        temperature=0.2
    )
    print(p)
    print(response.choices[0].message.content)
```

API controls you should know

These are knobs for controlling a probabilistic engine

- temperature → randomness / variability
- max_tokens → maximum output length and cost control
- top_p → probability range used for token sampling (light intro only)
- messages → the full structured prompt sent to the model

API control demo

Use one small demo to show controllable parameters.

```
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "Explain AI"}],
    temperature=0.8,
    max_tokens=50
)

print(response.choices[0].message.content)
print(response.usage)
```

LLMs are stateless... but systems feel stateful

This sounds contradictory — and that is the point

- The model does not remember anything between independent API calls
- If you do not resend context, the model cannot use prior context
- Chat systems feel stateful because the system replays history
- State is created by context accumulation, not by hidden memory inside the model

With history vs without history

This is the clearest live demo in the class.

```
# Without history
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[{"role": "user", "content": "What is my name?"}]
)

# With history
response = client.chat.completions.create(
    model="gpt-4o-mini",
    messages=[
        {"role": "user", "content": "My name is Ankit"},
        {"role": "assistant", "content": "Nice to meet you, Ankit"},
        {"role": "user", "content": "What is my name?"}
    ]
)
```

The hidden problem: context explosion

If we keep adding history, tokens, cost, and latency all increase

- More history = larger prompt_tokens
- Larger prompts = higher cost and slower responses
- Too much context can dilute the important signal
- Good systems manage context intelligently instead of sending everything

Prompt templates = scalable prompting

Professionals do not handcraft every prompt

- Templates make prompts reusable
- Templates make prompts dynamic
- Templates keep prompt structure consistent across many inputs
- This is where prompting starts to feel like engineering

```
template = """
Explain {topic} in exactly {length} sentence(s)
for {audience}.
"""

prompt = template.format(
    topic="hallucination",
    length=1,
    audience="software engineers"
)
```

Project upgrade: AI Knowledge Assistant — Version 2

This is the progression-compatible homework

- Add a structured prompt instead of a vague one
- Add a system role for behavior control
- Add temperature input so students can compare outputs
- Keep this same codebase — we will evolve it in future weeks

The model only knows what you send.

Structure improves quality. Systems scale through better architecture and better context management.